Realm

Persistencia de datos

Realm

La forma tradicional de conseguir la persistencia de datos en una aplicación es utilizar una base de datos clásica como podría ser MySQL, SQL Server o SQLite

Generalmente la conexión a una base de datos no es directa desde la aplicación sino que se utiliza una capa intermedia que hace de traducción entre las clases propias de una aplicación a las tablas propias de un base de datos

Esta capa intermedia se suele denominar ORM (Mapeo Objeto-Relacional)

Esta capa intermedia nos permite comunicarnos con la base de datos utilizando funciones en lugar de sentencias SQL

Realm

Aprovechando que esta forma de proceder la utilizareis en Acceso a Datos, lo que vamos a ver es una tecnología más nueva llamada Realm

Realm es una base de datos escrita en C++ pensada expresamente para dispositivos móviles

Con esta tecnología nos abstraemos de utilizar estos ORM o directamente la base de datos

Realm

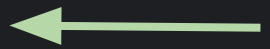
Ventajas

- Rapidez
- Eventos Listener -> Al cambiar un registro se pueden lanzar un evento
- Poca configuración
- Dispone de adaptadores para ListView o ReciclerView
- Encriptación
- Fácil definición de relaciones 1-n n-m
- TODO se define en las clases de nuestro modelo
- API bastante limpia y completa
- Multiplataforma

Realm

En el documento **build.gradle Project** añadiremos en las dependencias y en los repositorios el siguiente código:

```
buildscript {
    repositories {
        mavenCentral()
    }
    dependencies {
        classpath("com.android.tools.build:gradle:8.0.2" )
        classpath ("io.realm:realm-gradle-plugin:10.15.1" )
    }
}
plugins {
    alias(libs.plugins.android.application) apply false
}
```



Realm

En el documento **build.gradle Module** añadiremos en las dependencias este código:

```
plugins {
    alias(libs.plugins.android.application)
    id ("realm-android")
}
```

También sustituiremos los sdk en el bloque android

```
android {
    namespace = "com.ibansarria.examen1t2627"
    compileSdk = 36

    defaultConfig {
        applicationId = "com.ibansarria.examen1t2627"
        minSdk = 24
        targetSdk = 34
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner =
"androidx.test.runner.AndroidJUnitRunner"
    }
```

Realm

En el documento **build.gradle Module** también sustituiremos el bloque buildTypes

```
buildTypes {
    release {
        isMinifyEnabled = false
        proguardFiles(

getDefaultProguardFile("proguard-android-opti
mize.txt"),
            "proguard-rules.pro"
        )
    }
}
```

Realm

En el documento **gradle.properties** añadiremos al final la lína

```
android.useAndroidX=true
```

En el documento **gradle-wrapper.properties** cambiamos la versión de AGP

```
distributionPath=wrapper/dists
distributionUrl=https\://services.gradle.org/distributions/gradle-8.12-bin.zip
networkTimeout=10000
```



En el documento **libs.versions.toml** cambiamos la versión de AGP

```
agp = "8.9.1"
```

Realm

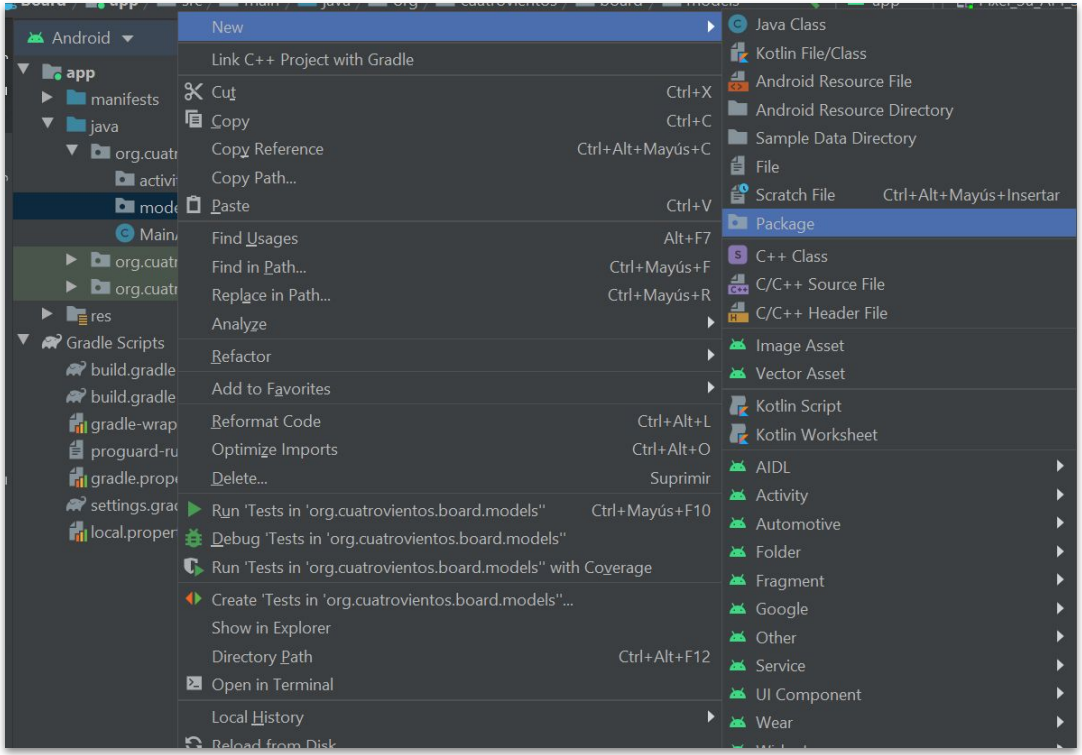
Creación de modelos

Ahora que nuestras aplicaciones van a empezar a ser un poco más complejas, vamos a estructurar nuestra aplicación en paquetes.

Desde la carpeta donde esta alojado el MainActivity, crearemos 4 paquetes:

- activities
- model
- app
- adapters

Realm


Android
app
manifests
java
org.cuatr
activi
mode
Main
org.cuatr
org.cuatr
res
Gradle Scripts
build.gradle
build.gradle
gradle-wrap
proguard-ru
gradle.prop
settings.gra
local.proper
New
Link C++ Project with Gradle
Cut Ctrl+X
Copy Ctrl+C
Copy Reference Ctrl+Alt+Mayús+C
Copy Path...
Paste Ctrl+V
Find Usages Alt+F7
Find in Path... Ctrl+Mayús+F
Replace in Path... Ctrl+Mayús+R
Analyze
Refactor
Add to Favorites
Reformat Code Ctrl+Alt+L
Optimize Imports Ctrl+Alt+O
Delete... Suprimir
Run 'Tests in 'org.cuatrovientos.board.models'' Ctrl+Mayús+F10
Debug 'Tests in 'org.cuatrovientos.board.models''
Run 'Tests in 'org.cuatrovientos.board.models'' with Coverage
Create 'Tests in 'org.cuatrovientos.board.models''...
Show in Explorer
Directory Path Ctrl+Alt+F12
Open in Terminal
Local History
Reload from Disk
Java Class
Kotlin File/Class
Android Resource File
Android Resource Directory
Sample Data Directory
File
Scratch File Ctrl+Alt+Mayús+Insertar
Package
C++ Class
C/C++ Source File
C/C++ Header File
Image Asset
Vector Asset
Kotlin Script
Kotlin Worksheet
AIDL
Activity
Automotive
Folder
Fragment
Google
Other
Service
UI Component
Wear

Realm

Creación de modelos

Dentro de nuestro paquete *models* crearemos las clases que vamos a necesitar dentro de nuestra aplicación

Estas clases deberán heredar de *RealmObject* para que Realm la trate con una tabla de la base de datos

```
public class Note extends RealmObject {}
```

Realm

Creación de modelos

Creamos nuestras clases de forma normal con la única particularidad de que *Realm* necesita de que creemos nosotros un constructor vacío.

Veremos más adelante como ir autogenerando el id, *Realm* no lo hace automáticamente

```
public class Note extends RealmObject {

    private int id;
    private String description;
    private Date createdAt;

    public Note(){

    }

    public Note (String description){
        this.id=0;
        this.description = description;
        this.createdAt = new Date();
    }
```

Realm

Creación de modelos

Para hacerle saber a *Realm* características propias de la base de datos utilizaremos anotaciones Java

@PrimaryKey
@Required

De esta forma conseguimos que en una misma clase este la representación Java para la aplicación y los datos necesarios para hacer de esa clase una tabla de la base de datos

```
public class Notes extends RealmObject {  80 usages  1 inheritor
    @PrimaryKey  2 usages
    private int id;
    private String description;  3 usages
    private Date createdAt;  2 usages

    public Notes(){  3 usages

    }
    public Notes(String description){  1 usage
        this.id = MyApplication.noteId.incrementAndGet();
        this.description =description;
        this.createdAt = new Date();
    }
```

Realm

Autoincremento de Id

Lo que necesitamos hacer para crear este autoincremente es crear una clase expresamente creada para albergar configuraciones y que se lance **antes de nuestros *Activities***

Lo que vamos a hacer es crear dentro del paquete *app* una clase Java que extienda de *Aplication*

```
public class MyApplication extends Application  {}
```

Cambiaremos también el *Manifest* para indicar que ese java es parte de la app

```
android:name=".app.MyApplication"
```

Sobreescribiremos su método onCreate para añadir en él todas las configuraciones que necesitemos

Realm

Autoincremento de Id

Lo que vamos a hacer nosotros en este punto del código es actualizar los *id* de todas nuestras clases consultandolos en las tablas

Una vez actualizados estos *id* al lanzar la aplicación, ya podríamos retocar nuestros constructores de clases para que el *id* se actualice automáticamente al crear un nuevo objeto

En esta clase MyAplication utilizaremos como tipo de dato para los *id* los AtomicInteger que disponen de un método incremantAndGet(), que nos facilitará la programación.

```
public static AtomicInteger noteID= new AtomicInteger();
```

Este dato será accesible desde toda la aplicación y lo utilizaremos a la hora de incrementar el *id*

Realm

Autoincremento de Id

Lo siguiente que haremos es crear un método que nos ayude a recuperar los *id*

Haremos uso de las clases genéricas que es un concepto nuevo

```
private <T extends RealmObject> AtomicInteger getIdByTable(Realm realm, Class<T>
anyClass){
```

- `<T extends RealmObject>` :Con esto indicamos que T será cualquier clase que herede de RealmObject, es decir cualquier clase que se corresponda con una tabla en la BBDD
- Este método devolverá un AtomicInteger que se corresponderá al último *id* guardado en la BBDD
- A este método se le pasará como parámetro la BBDD (`Realm realm`) y una clase genérica T (`Class<T> anyClass`) que como hemos indicado antes debe heredar de RealmObject

Realm

Autoincremento de Id

La forma de hacer esto es sencilla, nos servirá para ver cómo se hacen las consultas con Realm

Sacaremos todos los resultados de la tabla, un select *, podéis observar que se devuelven en una lista

```
RealmResults<T> results = realm.where(anyClass).findAll();
```

Después haremos una alternativa, si la lista de resultados está vacía devolveremos un AtomicInteger con valor 0 y si no está vacía el mayor *id* que esté guardado en la BBDD

```
private <T extends RealmObject> AtomicInteger  getIdByTable(Realm realm, Class<T>
anyClass){
   RealmResults<T> results = realm.where(anyClass).findAll();
   if (results.size()>0){
       return new AtomicInteger(results.max("id").intValue());
   }else{
       return  new AtomicInteger(0);
   }
}
```

Realm

Autoincremento de Id

En el método *onCreate* actualizaremos los valores de nuestros atributos que guardan los *id* ayudándonos del método que acabamos de crear

Además, construiremos el objeto de tipo Realm que no es la BBDD en si, sino una configuración que nos permite acceder a la BBDD

```
public void onCreate() {
    super.onCreate();
    setUpRealmConfig();
    Realm realm = Realm.getDefaultInstance();
    noteId = getIdByTable(realm, Notes.class);
    realm.close();
}
```

Realm

Autoincremento de Id

El último paso es volver a nuestros modelos y hacer que los identificadores se autoincrementen al crear un objeto.

Utilizaremos el método que hemos comentado previamente que obtiene y a la vez incrementa

```
this.id= MyApplication.noteID.incrementAndGet();
```

Al haber hecho estos datos de tipo *Static* y *Public* conseguimos que sean accesibles desde cualquier parte de la aplicación y que además no tengamos que crear un objeto de la clase *MyAplication* para poder acceder a ellos

Realm

Configuración inicial de la BBDD

Utilizaremos también esta clase *MyApplication* para la configuración inicial, en ella crearemos un método privado *setUpRealmConfig* con el siguiente código al que llamaremos desde el *onCreate* antes de crear el objeto *Realm*

```
private void setUpRealmConfig() {
    Realm.init(getApplicationContext());
    RealmConfiguration config =  new RealmConfiguration
            .Builder()
            .deleteRealmIfMigrationNeeded()
            .build();
    Realm.setDefaultConfiguration(config);
}
```

Realm

Transacciones

Lo primero que necesitamos es crear un objeto de tipo *Realm* en la *Activity* donde vamos a trabajar

Lo hemos creado ya en la configuración inicial para obtener los *id* pero para conseguir una programación limpia lo hemos destruido con un *close*

En las *Activities* en los que trabajaredemos con la BBDD deberemos tener una propiedad *Realm* que inicializaremos con la configuración que hemos definido en la configuración inicial

```
realm = Realm.getDefaultInstance();
```

Realm

Transacciones

El procedimiento para introducir una fila en una tabla es muy sencillo ya que los propios objetos *Realm* tienen en su definición las características de su tabla

Simplemente tendremos que crear el objeto en cuestión

```
Note note= new Note("myNote");
```

Y llamar al al método *copyToRealm* de la BBDD

```
 realm.copyToRealm(note);
```

Realm

Transacciones

Para que las transacciones con la BBDD se hagan de manera segura

Deberemos introducir todas estas operaciones entre dos llamadas a función que harán que todas las transacciones entre ellas se ejecuten solo si no ha ocurrido ningún error en el proceso para no dañar la BBDD

```
realm.beginTransaction();
Note note = new Note("Hola 2DAM");
realm.copyToRealm(note);
realm.commitTransaction();
```

Realm

Transacciones

En el caso que la transacción que vayamos a hacer no sea una inserción sino que sea una actualización la función a la que llamaremos será la siguiente

```
realm.copyToRealmOrUpdate(note);
```

Realm

Consulta

Vamos a ver la consulta más sencilla, para el resto de consultas deberéis utilizar la documentación oficial

Lo primero que necesitamos es un lugar donde guardar nuestra la consulta

Realm dispone de un tipo de dato especial, muy similar a un *ArrayList* para guardar los resultados

```
private RealmResults<Note> notes;
```

Realm

Consulta

La consulta más sencilla sería un select all de una tabla, se haría de la siguiente manera

```
notes= realm.where(Note.class).findAll();
```

De esta manera *Realm* transforma una línea de la base de datos en un objeto que se almacena en la lista

Si lo que queremos es obtener un solo valor lo podemos hacer de la siguiente manera

```
note= realm.where(Note.class).equalTo("id",noteId).findFirst();
```

Realm

Eliminar

Para eliminar un elemento de la base simplemente tenemos que llamar al método *deleteFromRealm* de ese objeto

```
note.deleteFromRealm();
```

Si por el contrario queremos hacer un borrado masivo podemos hacerlo llamando al metodo *deteAllFromRealm* de una lista de resultados (*RealmResults*) o de una lista de objetos que hereden de *Realm*

```
notes.deleteAllFromRealm();
```

Realm

Listener

Realm nos permite reaccionar a un evento de cambio

Si añadimos un listener a *RealmResults* u otro objeto heredado de *Realm* podremos reaccionar cada vez que haya una inserción o eliminación de la BBDD

Dentro del método *onChange* ejecutaremos el código que queramos tras el cambio.

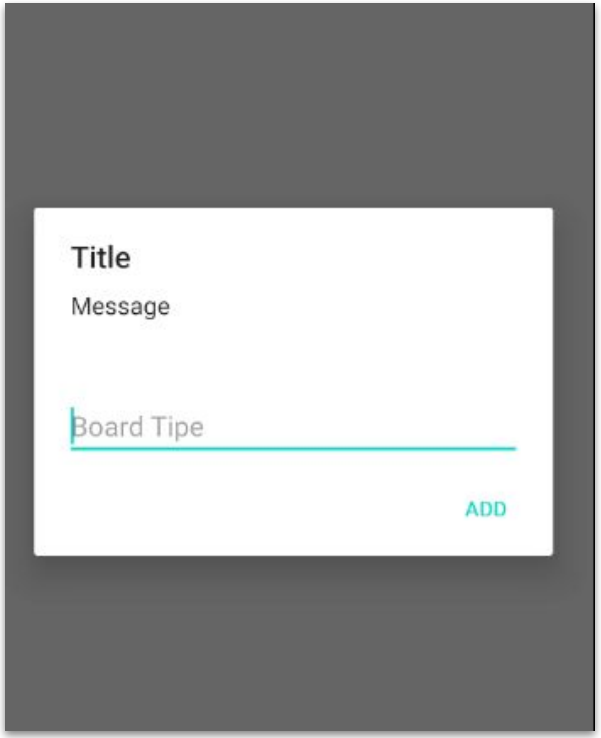
```
results.addChangeListener( new
RealmChangeListener<RealmResults<Note>>() {
    @Override
    public void onChange(RealmResults<Note> note) {
        adapter.notifyDataSetChanged();
    }
});
```

Realm

Pantalla de alerta (Documentación)

Para generar un mensaje de alerta lo primero que tenemos que hacer es generar un *layout* para esta pantalla

El título y el mensaje son una propiedad de la pantalla de alerta por lo que no van dentro del *layout*



Realm

Pantalla de alerta

Para crear el mensaje propiamente dicho crearemos un objeto del tipo *AlertDialog.Builder* que encargará de crear la pantalla

```
AlertDialog.Builder builder = new AlertDialog.Builder(this);
```

Después necesitaremos inflar una *View* con el *layout*

```
View viewInflated =
LayoutInflater.from(this).inflate(R.layout.dialog_create_board,null);
```

Después asignaremos al *View* al builder

```
builder.setView(viewInflated);
```

El título y el mensaje son una propiedad de la pantalla de alerta por lo que no van dentro del *layout*

Realm

Pantalla de alerta

Podemos modificar su título y mensaje a través del builder

```
builder.setTitle("title");
```

```
builder.setMessage("Message");
```

Como último paso crearemos un objeto de tipo *AlertDialog* utilizando el builder y lanzando su método create

```
 AlertDialog dialog = builder.create();
```

Este último paso crea la ventana pero no la muestra, para mostrarla tendremos que llamar al método *show*

```
 dialog.show();
```

Realm

Pantalla de alerta

De esta manera creamos la ventana pero lo interesante es poder interactuar con ella

Lo tendremos que hacer a través de la vista inflada

```
 EditText input = (EditText)
viewInflated.findViewById(R.id.editTextNewBoard);
```

Una vez que lo tenemos guardado en un objeto ya lo podríamos manipular

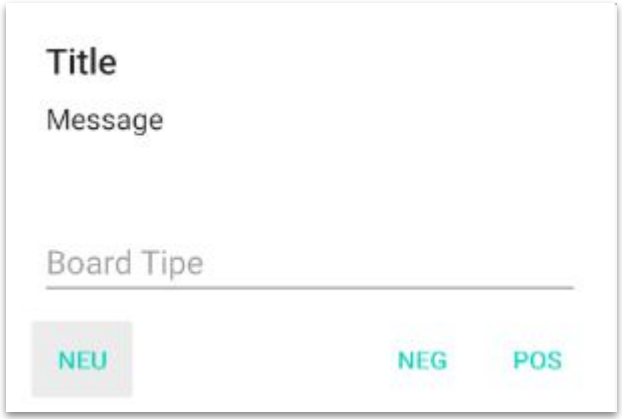
```
 input.setText("Hola Mundo");
```

Realm

Pantalla de alerta

Por último, a estas pantallas podemos añadir tres botones que funcionan de la misma manera que un botón normal.

Estos botones pueden ser positivo, negativo o neutral y esto indica sólamente su posición en la pantalla



Realm

Pantalla de alerta

Para añadir estos botones llamaremos a un método del builder que recibirá el texto a mostrar y un listener como el resto de botones

```
builder.setPositiveButton("pos", new DialogInterface.OnClickListener() {...}
```

```
builder.setNegativeButton("neg", new DialogInterface.OnClickListener() {...}
```

```
builder.setNeutralButton("neu", new DialogInterface.OnClickListener() {...}
```

Realm

Option menu

Context menu